

# Final Project: Create a Learning Dashboard using Bolt



**Estimated time:** 1 hour

Welcome! As part of your final project, your organization's CodeCraftHub learning platform needs a course management interface. You will use Bolt, an AI-powered UI generation tool, to build a functional frontend that allows users to **create, read, update, and delete courses** through your backend service.

This project focuses on creating a complete CRUD (Create, Read, Update, Delete) interface using basic HTML, CSS, and JavaScript that will seamlessly connect to the backend code that you completed in Part 1 of this final project

You can choose from the following two Bolt environments to complete your UI tasks.

- **Bolt.new** (A SaaS platform with free limited tokens): Open the Bolt website using the following link: <https://bolt.new>
- **Bolt.diy** (An embedded Generative AI studio environment provided as part of this course). You'll find access to this stuido environment displayed as a separate lab within your course.

## Important!

Do not close this lab environment. Keep this environment lab open in its separate window to use its instructions within the bolt.diy environment. You will also return this lab for additional instructions after building the project's functional frontend.

## Learning objectives

After completing this lab, you will be able to perform the following tasks:

- Generate user interfaces using Bolt with effective prompting
- Create a complete CRUD dashboard using HTML, CSS, and JavaScript
- Connect frontend UI to backend REST APIs
- Implement create, read, update, and delete operations in a single-page application
- Work within token constraints of AI code generation tools

## Prerequisites

Before you begin, review the following lab requirements:

- You have a basic understanding of HTML, CSS, and JavaScript.
- You completed Part 1 of the final project (CodeCraftHub backend lab) in the programming language of your choice (JavaScript/Python/Java), and have a running Service with the following endpoints:
  - POST /api/courses (Create)
  - GET /api/courses (Read all)
  - GET /api/courses/{id} (Read one)
  - PUT /api/courses/{id} (Update)
  - DELETE /api/courses/{id} (Delete)
- If you are using Bolt.new, you should have completed a previous lab and have access to <https://bolt.new>. The free tier is sufficient for this lab.
- Your backend service is running and accessible in your environment.

**Note:** While this lab aims to be as up-to-date as possible about token information, Bolt can update the features and available tokens provided with their free plan at any time.

## Your primary tasks

- You will create a simple, functional CRUD dashboard
- You will focus on the dashboard's core functionality: Managing courses
- You will use a single, well-crafted prompt to generate everything at once

## Project features

- A dashboard that displays all courses
- A form to add new courses
- Ability to edit existing courses
- Ability to delete courses

- Basic HTML, CSS, JavaScript only (no frameworks)
- API calls for all CRUD operations
- Clean, minimal styling
- A responsive layout

## Verify that your backend is running

The API base URL depends on which backend you implemented:

### JavaScript/Node.js backend:

- Default Port: 5000
- Base API URL: `http://localhost:5000/api/courses`

### Python backend:

- Default Port: 5000
- Base API URL: `http://localhost:5000/api/courses`

### Java/Spring backend:

- Default Port: 8080
- Base API URL: `http://localhost:8080/api/courses`

**Note:** Ensure your backend is running before generating and testing the dashboard.

## Generate your dashboard with Bolt

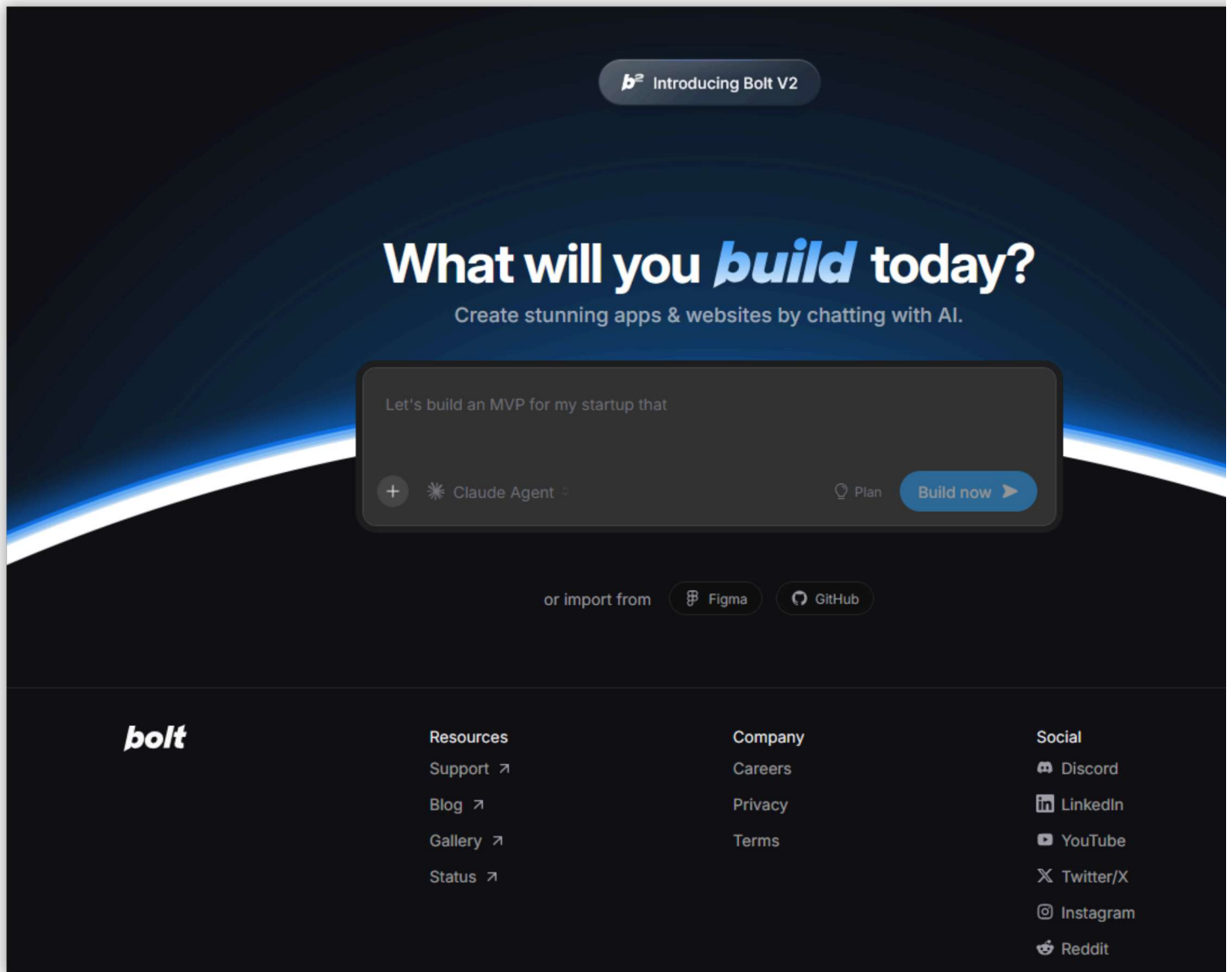
Before continuing, choose whether to use Bolt.new or Bolt.diy. After making your choice, follow the corresponding section to generate the UI.

- Bolt.new is a commercial tool and offers limited AI generation on the free tier. You will be responsible for any additional costs.
- Bolt.diy is a free, course-provided tool, which also has generation limits, but no extra charges apply.

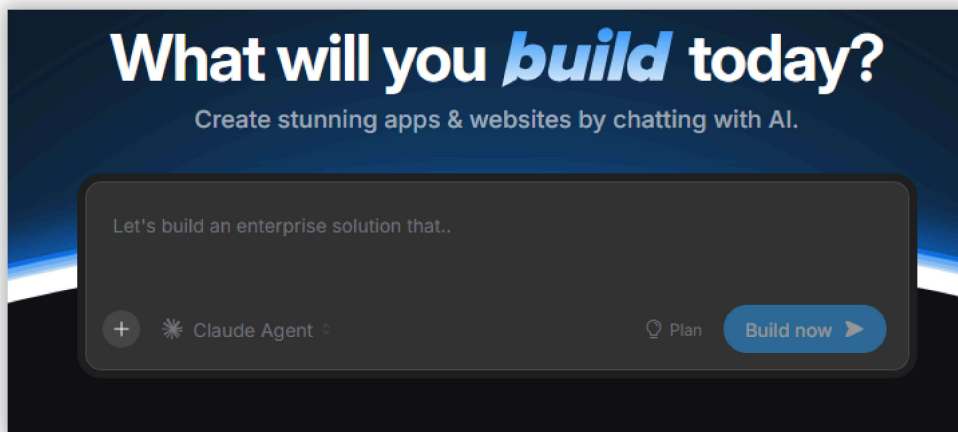
## [Option A] Using Bolt.new

If you decided to use Bolt.new to generate your complete CRUD dashboard in ONE prompt, follow the below instructions:

1. Open your web browser (Chrome works best) and navigate to <https://bolt.new>.



2. You will see a chat interface where you can describe what you want to build.



### The single prompt strategy

Because tokens are limited, you'll use **one** comprehensive prompt to generate everything. This prompt is carefully crafted to be token-efficient while producing a complete, functional dashboard.

### Prompt template

Copy this prompt and customize the “YOUR\_BACKEND\_URL” section as described below in the prompt:

### Customize this prompt:

Replace [YOUR\_BACKEND\_URL] with:

- `http://localhost:5000` for Node.js or Python
- `http://localhost:8080` for Java

### Example for Node.js/Python backend:

- Base API URL: `http://localhost:5000/api/courses`

### Example for Java backend:

- Base API URL: `http://localhost:8080/api/courses`

Create a learning management dashboard with API operations.

Requirements:

1. Single HTML file with embedded CSS and JavaScript
2. Display all courses in a table/card layout
3. Include a form to add new courses
4. Allow editing courses inline or in a modal
5. Include delete buttons for each course
6. Use vanilla JavaScript (no frameworks like React)
7. Use basic CSS for styling (no frameworks like Bootstrap)
8. Generate only one HTML file with all JS and CSS
9. Focus on calling backend APIs
10. Do not use any database

Technical details:

- Base API URL: [YOUR\_BACKEND\_URL]/api/courses
- All API calls should handle loading states and errors

API endpoints to implement:

1. GET /api/courses - fetch all courses
2. POST /api/courses - create new course (body: name, description, target\_date, status)
3. PUT /api/courses/{id} - update course (body: any fields to update)
4. DELETE /api/courses/{id} - delete course

Course fields:

- id (read-only, auto-generated by backend)
- name (required, text input)
- description (required, textarea)
- target\_date (required, date input in YYYY-MM-DD format)
- status (required, dropdown with: "Not Started", "In Progress", "Completed")
- created\_at (read-only, displayed as formatted date)

Design:

- Title/Header with "CodeCraftHub: Your Learning Management Platform"
- Add Course form at the top with all fields
- Course list below showing all courses
- Each course should have Edit and Remove buttons
- Use a simple, clean table or card layout
- Show loading spinner while fetching/saving data
- Show success/error messages for all operations
- Make it responsive for mobile devices

Colors:

- Use a professional purple/amber theme:
  - #8B5CF6 for primary
  - #F59E0B for success
  - #EF4444 for delete

Error handling:

- Show error if API calls fail
- Validate all required fields before submission
- Show user-friendly error messages

## Generate the code

1. Copy your customized prompt.
2. Paste the prompt into bolt.new.
3. Wait for the generation to complete (usually 3–5 minutes).
4. Review the generated code.
5. Download or copy the generated code.
6. Proceed to the lab section: “Export and run your dashboard”.

## [Option B] Using Bolt.diy

If you decided to use Bolt.diy to generate your complete CRUD dashboard in ONE prompt, follow the below instructions:

1. Locate and open the Access bolt.diy lab link in your course platform. You’ll find the lab link immediately following this lab.
2. You will see a chat interface where you can paste the following prompt to instruct what you want to build.
3. Keep this lab open so that you can use the following instructions to generate your front-end project code.

### The single prompt strategy

Because tokens are limited, you’ll use **one** comprehensive prompt to generate everything. This prompt is carefully crafted to be token-efficient while producing a complete, functional dashboard.

### Prompt template

Copy this prompt and customize the “YOUR\_BACKEND\_URL” section as described below in the prompt:

#### Customize this prompt:

Replace [YOUR\_BACKEND\_URL] with:

- `http://localhost:5000` for Node.js or Python
- `http://localhost:8080` for Java

#### Example for Node.js/Python backend:

- Base API URL: `http://localhost:5000/api/courses`

#### Example for Java backend:

- Base API URL: `http://localhost:8080/api/courses`

```
Create a learning management dashboard with API operations.
Requirements:
1. Single HTML file with embedded CSS and JavaScript
2. Display all courses in a table/card layout
3. Include a form to add new courses
4. Allow editing courses inline or in a modal
5. Include delete buttons for each course
6. Use vanilla JavaScript (no frameworks like React)
7. Use basic CSS for styling (no frameworks like Bootstrap)
8. Generate only one HTML file with all JS and CSS
9. Focus on calling backend APIs
10. Do not use any database
Technical details:
- Base API URL: [YOUR_BACKEND_URL]/api/courses
- All API calls should handle loading states and errors
API endpoints to implement:
1. GET /api/courses - fetch all courses
2. POST /api/courses - create new course (body: name, description, target_date, status)
3. PUT /api/courses/{id} - update course (body: any fields to update)
```

```
4. DELETE /api/courses/{id} - delete course
Course fields:
- id (read-only, auto-generated by backend)
- name (required, text input)
- description (required, textarea)
- target_date (required, date input in YYYY-MM-DD format)
- status (required, dropdown with: "Not Started", "In Progress", "Completed")
- created_at (read-only, displayed as formatted date)
Design:
- Header with "CodeCraftHub: Your Learning Management Platform"
- Add Course form at the top with all fields
- Course list below showing all courses
- Each course should have Edit and Remove buttons
- Use a simple, clean table or card layout
- Show loading spinner while fetching/saving data
- Show success/error messages for all operations
- Make it responsive for mobile devices
Colors:
- Use a professional purple/amber theme:
  - #8B5CF6 for primary
  - #F59E0B for success
  - #EF4444 for delete
Error handling:
- Show error if API calls fail
- Validate all required fields before submission
- Show user-friendly error messages
```

## Generate the code

1. Copy your customized prompt.
2. Paste the prompt into bolt.new.
3. Wait for the generation to complete (usually 3–5 minutes).
4. Review the generated code.
5. Download or copy the generated code.
6. Proceed to the lab section: “Export and run your dashboard”.

## Export and run your dashboard

Whether you’ve used Bolt.new or Bolt.diy, copy the generated HTML from the respective tools and follow these instructions:

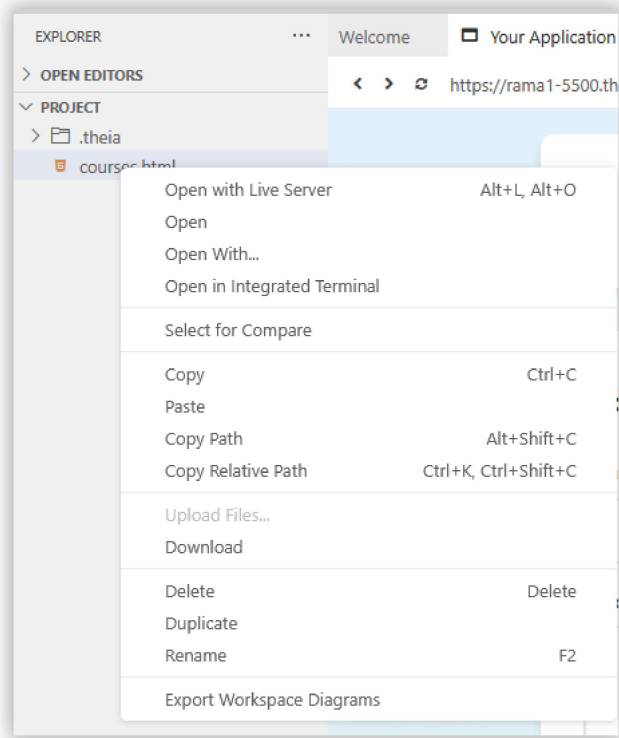
- Save the generated code as .html in the same folder as your backend code (completed in Part 1 of the final project).
- For example: The folder would be codecrafthub.

## Run your dashboard

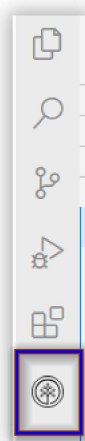
1. Open the project in your development environment.
2. Make sure your backend server is running.
3. Ensure the .html file has been downloaded and stored in the same folder as your backend code.

Next,

1. In the Explorer panel, locate the .html file.
2. Right-click the .html file and select “Open with Live Server”.

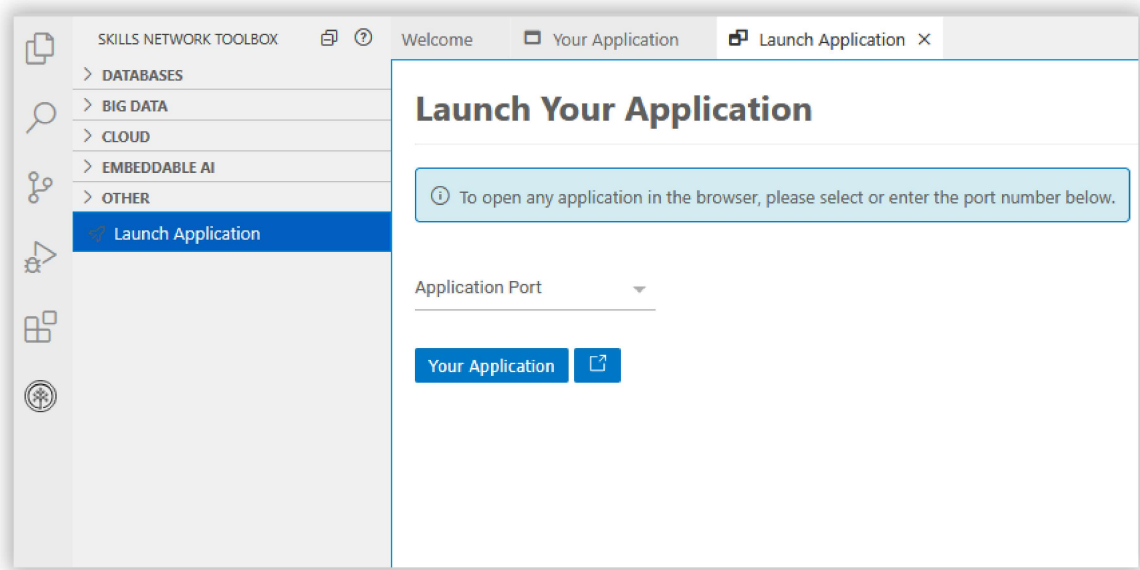


3. Your environment will display a popup with the port number. Make note of it. 4. Select the

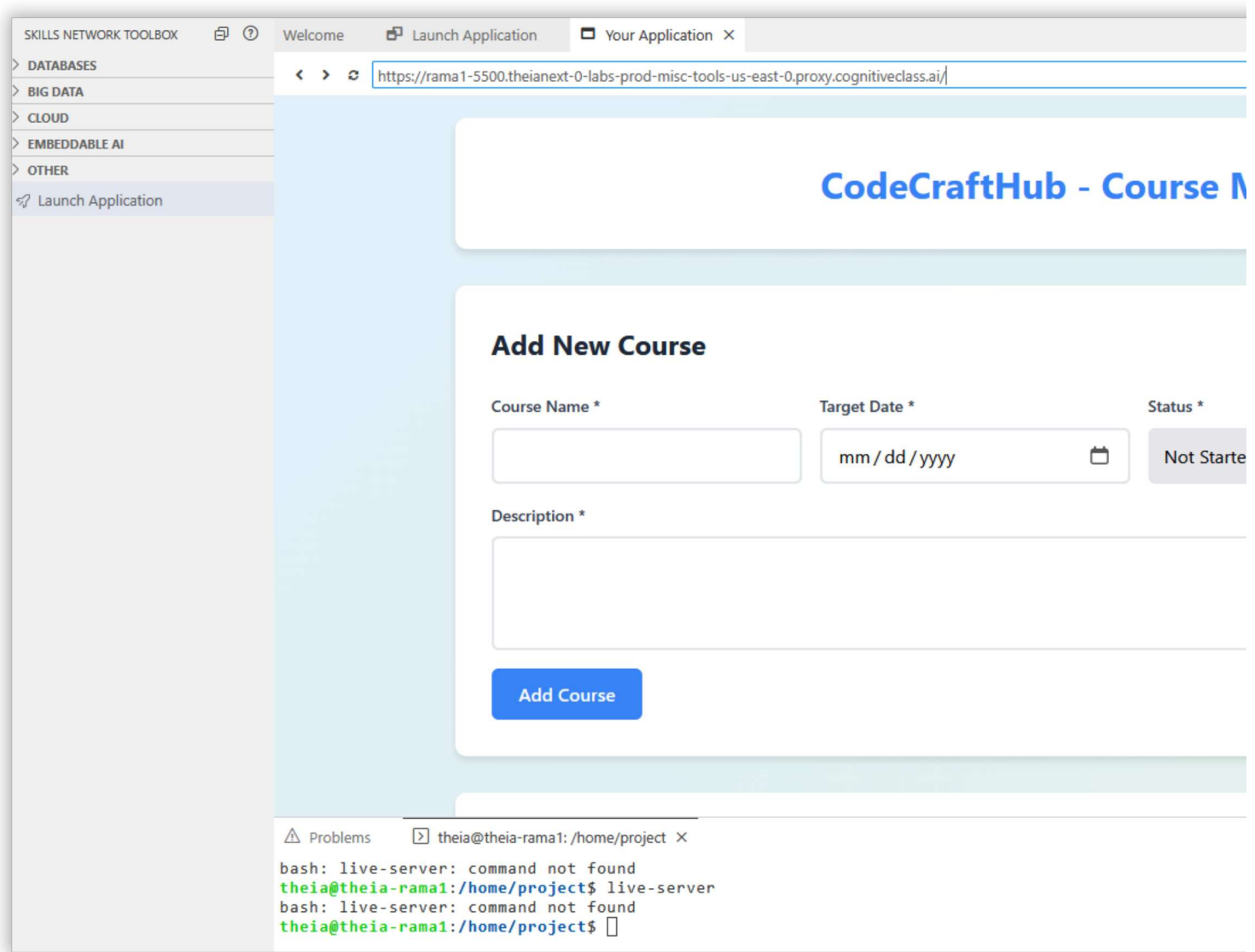


“Launch Application” icon on the left sidebar. provide the same port number that the environment displayed earlier.

5. On the “Launch Your Application” page, in the “Application Port” section,



6. Select the “Your Application” button, and you will see the running application.



## [Optional] Review the exemplar HTML code

Here is an example of a generated HTML code for your reference. Either double-click or right-click to download and save this [GenAI Generated HTML code](#) to your local machine or the destination of your choice.

You can review this example HTML code and compare the functions in this HTML example to your own HTML code.

## [Optional] Push your work to GitHub

Here's how to save your work to GitHub for future reference.

### Create the GitHub repository

1. Go to <https://github.com>.
2. Select **New repository**.
3. Name the repository: `codecrafthub-course-dashboard`.
4. Add the description: "CRUD dashboard for CodeCraftHub learning platform".
5. Make the repository public.
6. Select **Create repository**.

### Initialize Git in your project

```
# Navigate to your dashboard folder
cd /path/to/dashboard-folder
# Initialize git
```



```
git init
# Create the .gitignore file
echo ".DS_Store" > .gitignore
# Configure git (if not already done)
git config --global user.email "your.email@example.com"
git config --global user.name "Your Name"
# Add files
git add .
# Commit
git commit -m "Initial commit - CodeCraftHub CRUD dashboard"
# Connect to GitHub repository
git remote add origin https://github.com/yourusername/codecrafthub-course-dashboard.git
# Push to GitHub
git branch -M main
git push -u origin main
```

When prompted, provide the following credentials:

- **Username:** Your GitHub username
- **Password:** Your GitHub Personal Access Token (not your regular password)

**Note:** If you don't have a Personal Access Token, follow the [Generate GitHub personal access token](#) lab.

## Summary

You have successfully completed the CodeCraftHub Course Management Dashboard project. You:

- Used bolt.new or bolt.diy to generate a complete CRUD dashboard with a single, well-crafted prompt
- Created a functional UI using only HTML, CSS, and JavaScript
- Implemented all four CRUD operations (Create, Read, Update, and Delete)
- Integrated your frontend with backend REST APIs
- Created a responsive, user-friendly interface
- [Optional] Documented and version controlled your project with Git and GitHub

### Key takeaways

1. **API-agnostic design:** Your dashboard works with any backend (Node.js, Python, Java) as long as the API endpoints follow the same structure.
2. **CRUD operations:** You implemented the four fundamental operations for data management:
  - **Create:** Add new courses via POST
  - **Read:** Display all courses via GET
  - **Update:** Modify courses via PUT
  - **Delete:** Remove courses via DELETE
3. **Token efficiency:** You learned to use a comprehensive, single prompt to maximize bolt.new's free tier plan.

Congratulations on completing the CodeCraftHub Course Management Dashboard! You have successfully leveraged Bolt to create a fully functional CRUD interface that integrates seamlessly with your backend service!

**Note:** During assessment submission, you will be required to copy and paste portions of your generated HTML code to answer questions, so ensure the interface reflects the design elements and functionality specified in the prompt.

### Author(s)

[Ramanujam Srinivasan](<https://www.linkedin.com/in/ramanujamrs/>)

© IBM Corporation. All rights reserved.